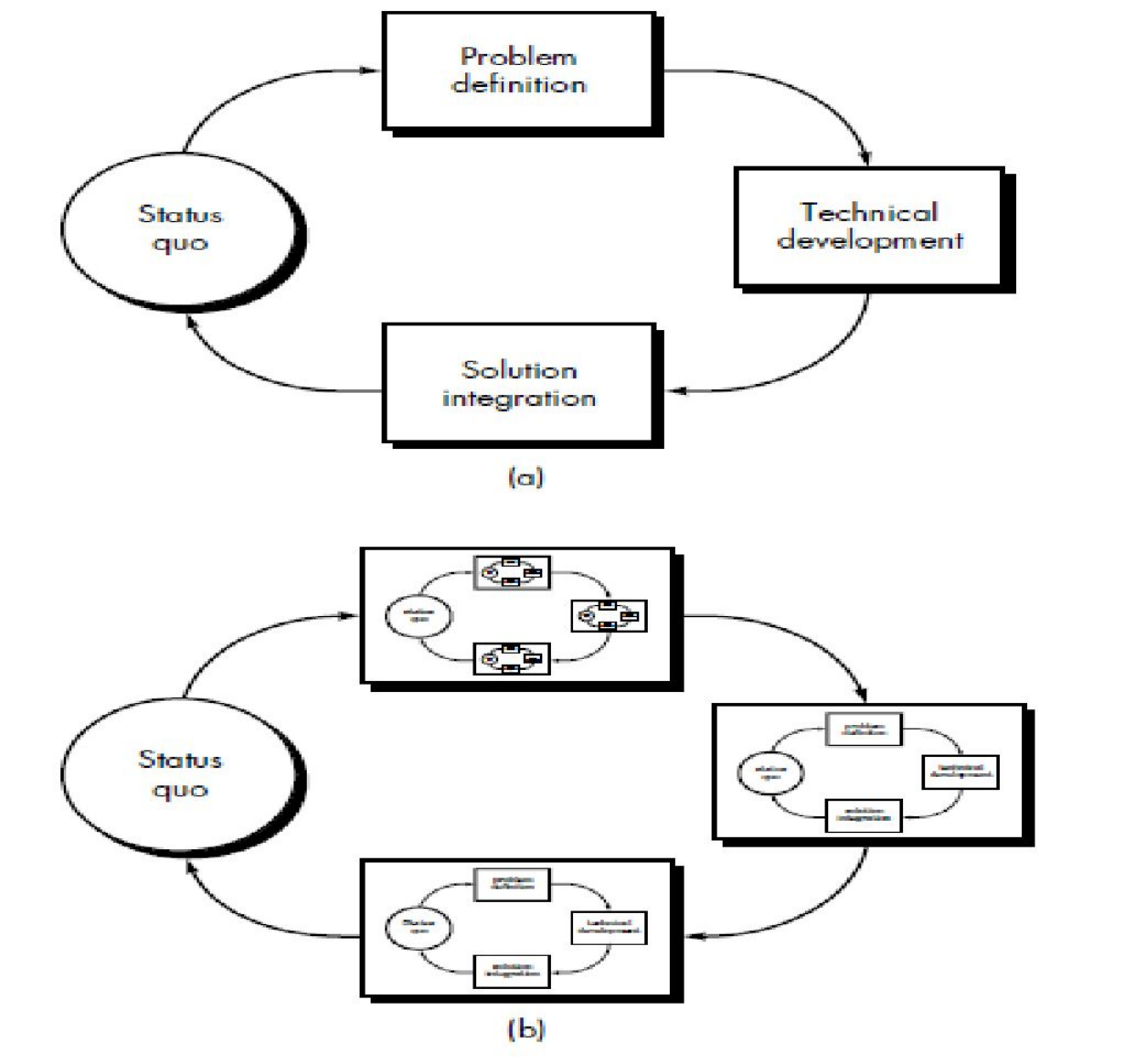
Software Engineering

Lecture Five

SOFTWARE PROCESS MODELS

To solve actual problems in an industry setting, a software engineer or a team of engineers must incorporate a development strategy that encompasses the process, methods, and tools layers and the generic phases This strategy is often referred to as a process model or a software engineering paradigm. A process model for software engineering is chosen based on the nature of the project and application, the methods and tools to be used, and the controls and deliverables that are required.



(a) The phases of a problem solving loop

(b) The phases within phases of the problem solving loop

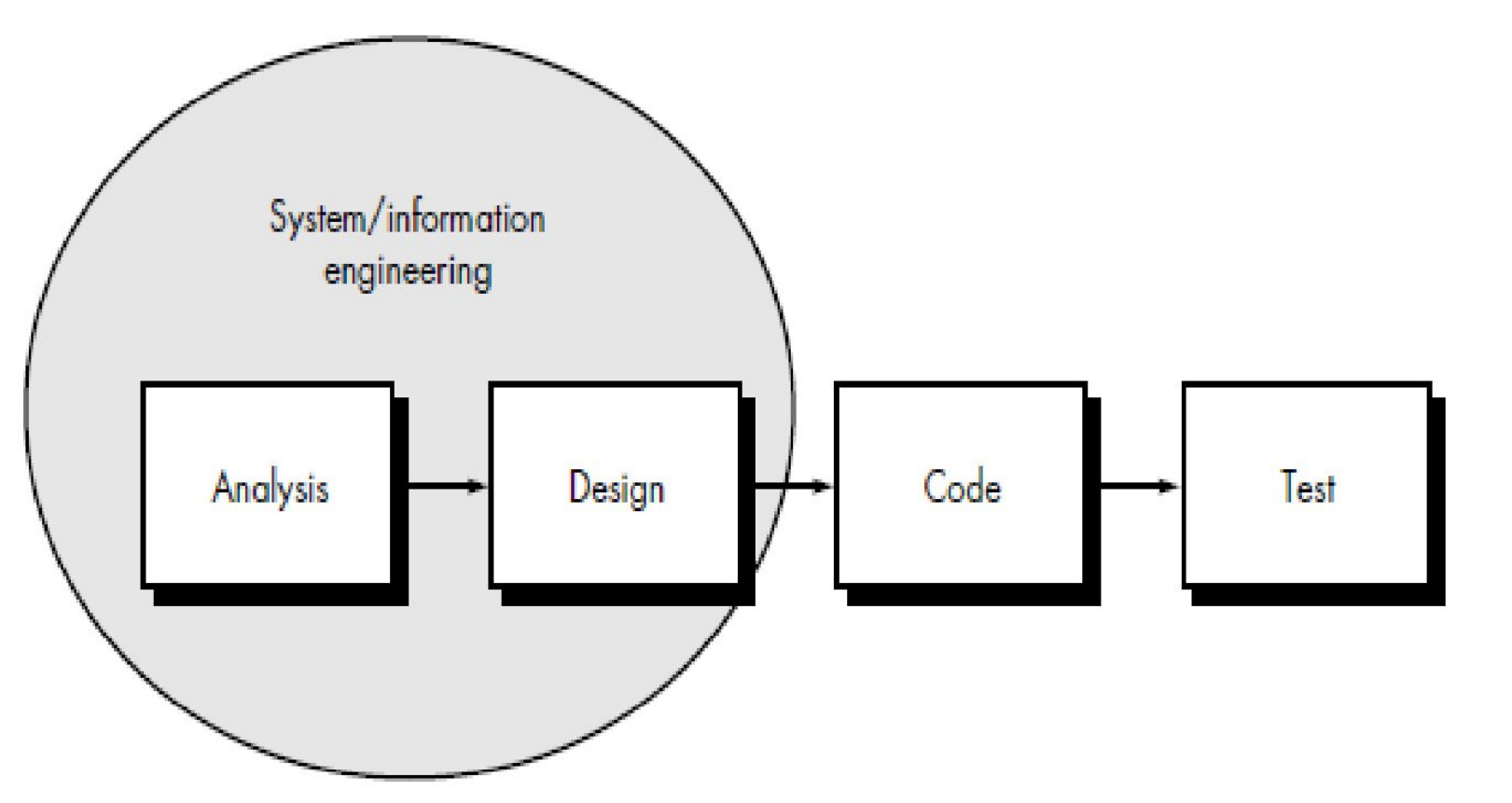
All software development can be characterized as a problem solving loop (Figure a) in which four distinct stages are encountered: status quo, problem definition, technical development, and solution integration. Status quo "represents the current state of affairs" ; problem definition identifies the specific problem to be solved; technical development solves the problem through the application of some technology, and solution integration delivers the results (e.g., documents, programs, data, new business function, new product) to those who requested the solution in the first place.

In Figure b, each stage in the problem solving loop contains an identical problem solving loop, which contains still another problem solving loop (this continues to some rational boundary; for software, a line of code).

THE LINEAR SEQUENTIAL MODEL

Sometimes called the classic life cycle or the waterfall model, the linear sequential model suggests a systematic, sequential approach to software development that begins at the system level and progresses through analysis, design, coding, testing, and support. Figure below illustrates the linear sequential model for software engineering. Modelled after a conventional engineering cycle, the linear sequential model encompasses the following activities:

The linear sequential model



- **System/information engineering and modeling**. Because software is always part of a larger system (or business), work begins by establishing requirements for all system elements and then allocating some subset of these requirements to software. This system view is essential when software must interact with other elements such as hardware, people, and databases.
- **Software requirements analysis**. The requirements gathering process is intensified and focused specifically on software. To understand the nature of the program(s) to be built, the software engineer must understand the information domain for the software, as well as required function, behaviour, performance, and interface. Requirements for both the system and the software are documented and reviewed with the customer.

- **Design**. Software design is a multistep process that focuses on four distinct attributes of a program: data structure, software architecture, interface representations, and procedural (algorithmic) detail. The design process translates requirements into a representation of the software that can be assessed for quality before coding begins.
- **Code generation**. The design must be translated into a machine-readable form. The code generation step performs this task.
- **Testing**. Once code has been generated, program testing begins. The testing process focuses on the logical internals of the software, ensuring that all statements have been tested, and on the functional externals

Real-World Example: Online Food Delivery System Development

- **Requirements (Analysis):** The project team gathers all requirements, such as user registration, menu display, payment, and driver tracking. All requirements are documented, and no changes are allowed once this phase concludes.
- **Design**: Architects create the system design based on requirements, deciding on technology, database structure, and user interface layout.
- **Implementation (Coding):** Developers build the system module by module, creating the application, databases, and functionality exactly as designed.
- **Testing:** The completed software is rigorously tested for bugs, security issues, and usability to ensure it meets the initial requirements.
- **Deployment:** The fully tested application is deployed to the market for users.
- **Maintenance:** The system is monitored, and updates are applied to fix bugs or improve performance as needed.

problems of the linear sequential model

- **Inflexibility to Change:** The linear, sequential nature makes it difficult to go back and change something in a previous phase (e.g., changing requirements after design has started).
- **High Risk and Uncertainty:** Because testing is only done at the end of the development cycle, bugs or design flaws are discovered very late, making them expensive and time-consuming to fix.
- **Poor Handling of Complex Projects:** It is not suitable for projects where requirements are not clearly defined or are likely to change, which is common in modern software development.

problems of the linear sequential model

- **No Working Software Until Late:** A functioning product is only produced at the very end of the lifecycle, meaning stakeholders cannot provide feedback early, leading to potential misalignment with user needs.
- **Limited Customer Involvement:** The model relies on heavy documentation rather than continuous customer collaboration, often resulting in a final product that does not meet user expectations.
- **Long Development Time:** The sequential, step-by-step approach can lead to longer overall development times compared to iterative approaches.